

Aprendizagem Aplicada à Segurança

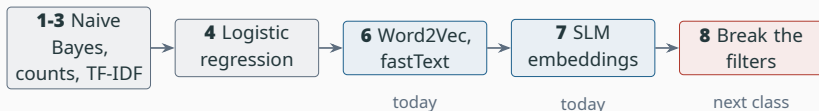
Class 3: SPAM Detection II, from Words to Meaning

Mário Antunes

October 2, 2026

Universidade de Aveiro

Where we stand



- Class 2 ended with a table of models within 0.94 to 0.96 F1, all seeing a message as **a bag of exact words**.
- Today: two questions. **Where do word and sentence vectors come from** (our data or pre-trained)? **How do we run a language model** we do not want inside the notebook?

Two blind spots of a bag of words

- **No similarity:** cash, money and prize are as unrelated as cash and banana.
- **No out-of-vocabulary handling:** fr33, fre3, freee are unknown words; their evidence is lost.

Message	Shared with claim your cash prize
collect your free money reward c4sh pr1ze, cl4im n0w	your only nothing

- A **vector** per word fixes the first: similar words get nearby vectors. **Sub-words** fix the second.

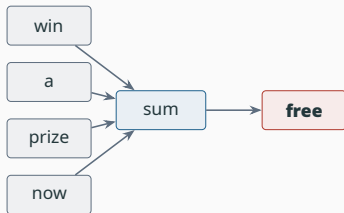
You shall know a word by the company it keeps

Words that occur in similar contexts have similar meanings. (distributional hypothesis, Harris 1954, Firth 1957)

- In spam, free appears next to entry, mobile, camera, txt, claim; in chat, home appears next to tonight, dinner, late.
- **Word2Vec** (Mikolov, Chen, et al. 2013) learns a dense vector per word (100 to 300 numbers) so that words with the **same neighbours** get the **same direction**.
- No labels needed: the text is the supervision. That matters, because **labels are scarce, text is cheap**.

Two ways to predict

CBOW: context $\rightarrow$ word



skip-gram: word $\rightarrow$ context



- **CBOW** averages the context to predict the word: fast, smooth for frequent words. **Skip-gram** predicts each context word from the word: better for rare words.

Training with negative sampling

For each observed pair (word w , context word c), push the score of the pair up and the score of k random words n_i down (Mikolov, Sutskever, et al. 2013):

$$\log \sigma(\mathbf{v}_w \cdot \mathbf{u}_c) + \sum_{i=1}^k \log \sigma(-\mathbf{v}_w \cdot \mathbf{u}_{n_i})$$

- $k = 5$ noise words instead of a softmax over the whole vocabulary: cost $O(k)$ per pair, not $O(|V|)$.
- Knobs: `vector_size`, `window(5)`, `min_count(2)`, `epochs(30)`, `sg`. Same logistic loss and descent as `lab 02_01`.

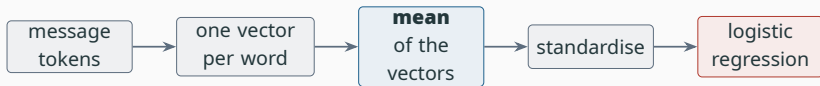
Vectors have geometry

Nearest neighbours by cosine similarity, vectors trained on **our** 65,000 tokens (skip-gram) and **Google News** (100 billion words):

word	in-domain (SMS)	pre-trained (news)
free	unlimited, camcorder, jamster, bluetooth	Free, FREE, complimentary
prize	caller, guaranteed, pound, bonus	prizes, award, grand_prize
home	lonely, chillin, knackered	house, homes, residence

- The news vectors know **English**. The in-domain vectors know **this corpus**: offers, phone-shop words, texting slang.

From words to a message



$$\mathbf{x} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}_{w_i} \in \mathbb{R}^d$$

- Words without a vector are skipped; **word order is lost** (not free and free not are the same message).
- The classifier learns $d + 1$ weights instead of one per word: a much smaller problem than TF-IDF's thousands.

Two sources of vectors

	In-domain	Pre-trained
Corpus	the 4,095 training messages (65,000 tokens)	Google News: 100 billion words; Common Crawl (fastText)
Vocabulary Domain	3,107 words (min count 2) exactly SMS: slang, £, phone numbers	3 million words (we load 200,000) news and web pages
Cost	seconds on a CPU	1.7 GB download; 22 MB for the compressed fastText
Labels	none	none

- Only the **training** text is used, never the test messages, never the labels.
- Which is better for a spam filter? The intuition says *pre-trained*: 1.5 million times more text.

What a news vocabulary does not know about SMS

Spam tokens without a Google News vector, by frequency: num (512), ! (345), msg (40), pobox (30), optout (17), sae, mobileupd, freemsg.

Vectors	test tokens without a vector
Word2Vec in-domain	9.1%
fastText in-domain (n-grams)	0.0%
Word2Vec Google News (200k words)	7.1%
fastText Common Crawl, compressed	5.2%

- 13.5% of all **spam** tokens have no Google News vector: phone numbers, exclamation marks and SMS jargon **are** the spam signal.

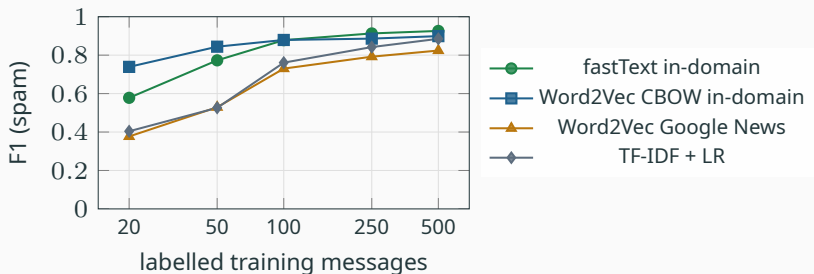
The result: in-domain wins

Mean of the word vectors + standardised logistic regression,
F1 on the 1,024 test messages:

Vectors	F1
fastText, in-domain	0.940
Word2Vec skip-gram, in-domain	0.935
Word2Vec CBOW, in-domain	0.912
Word2Vec Google News, pre-trained	0.888
fastText Common Crawl, pre-trained	0.882
TF-IDF + logistic regression (reference)	0.944

- Vectors learnt on the SMS beat vectors from 1.5 million times more text, and match TF-IDF with far fewer weights.

Few labels: unlabelled text is the resource

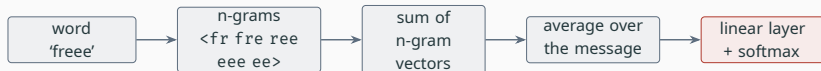


- The embeddings saw all the **unlabelled** training text; the classifier saw only n labels. TF-IDF has no unlabelled stage.

When does pre-training pay?

- **Coverage decides.** The pre-trained vectors are as good as the share of *your* vocabulary they contain: news vectors miss the SMS tokens that matter.
- **In-domain wins** when you have plenty of unlabelled text from the *same* stream (a mail gateway has millions of messages).
- **Pre-trained wins** when your own text is tiny or the domain is close to the training text (product reviews, news).
- The third way is what the next sections show: **pre-train on so much text that coverage stops being a problem**, then keep the model frozen (sentence embeddings).

Words are made of pieces



- A word vector is the **sum of the vectors of its character n-grams** (Bojanowski et al. 2017): freee shares most n-grams with free.
- Skip-gram over words *and* n-grams gives every word a vector, **also unseen ones**. The supervised variant is average-then-linear (Joulin et al. 2017).

Out-of-vocabulary words in practice

Nearest neighbours of disguised words
(notebooks/01-spam/05):

word	trained on SMS	pre-trained, compressed
freee congratulationz	free , freemsg, freefone congratulations , congrats	guilty, dioxide (wrong) congratulations , Congrats
txtt	txtno, txt , txts	zero vector
fr33	fr, frm, fri (misses free)	zero vector

- Sub-words help, they do not guarantee: fr33 is dominated by its <fr n-gram. The 22 MB compressed model **pruned** the n-grams it would need.

Supervised fastText

- Same architecture, trained end to end on the labels (word bigrams + character n-grams, average, linear layer, softmax): seconds on a CPU.

Model	Precision	Recall	F1
fastText, mean of in-domain vectors + LR	0.975	0.907	0.940
fastText supervised	0.983	0.915	0.948
TF-IDF words + LR	0.983	0.907	0.944
TF-IDF characters + LR	0.983	0.899	0.939

- On clean text: **on par** with a tuned linear model. The gain is speed, dense vectors and tolerance to spelling variation, not a higher score.

Static versus contextual

- A static vector has **one meaning per word**: bank in river bank and in bank transfer share a vector; so do call in call now and call me later.
- A **Transformer encoder** reads the whole message and outputs a vector for each token **that depends on the rest of the message**.
- A **sentence embedding** pools those vectors (mean, or the last token) into **one vector per message**.
- **Small language model (SLM)**: a Transformer up to about 1 billion parameters, small enough for a laptop CPU or an integrated GPU.

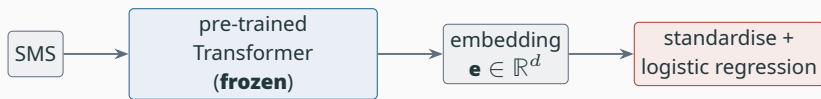
How an embedding model is trained

- **Contrastive learning** on billions of text pairs: pull a pair together, push the other texts of the batch apart (*InfoNCE* (Oord et al. 2018; Reimers and Gurevych 2019)).
- Similar meanings end up **close in cosine similarity**; normalised vectors: $\cos \theta = \mathbf{e}_1 \cdot \mathbf{e}_2$.

Pair of messages (Nomic v2)	cosine
a prize message and a rewrite with other words	0.76
the prize message and a chat message	0.41

- TF-IDF gives the first pair a similarity close to 0.

Frozen encoder, linear probe



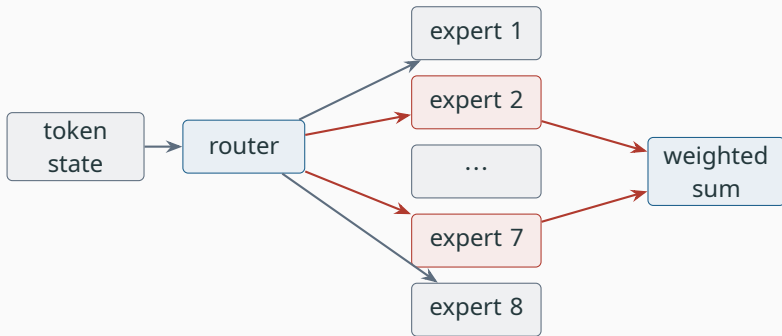
- Only $d + 1$ weights are trained: a *linear probe*.
Standardise each dimension first: LLM embeddings are **anisotropic** (all cosines high).
- No fine-tuning, no GPU for training. A small Keras MLP head on Nomic (JAX) gives 0.965 against 0.952: noise-level.

The 2026 zoo of small embedders

Model	Params	Dim.	Note
all-MiniLM-L6-v2	22 M	384	BERT-style, F16 GGUF 46 MB (Reimers and Gurevych 2019)
EmbeddingGemma	300 M		gated licence (Vera et al. 2025)
Nomic Embed v2 MoE	475 M (305 M active)	768	Matryoshka, Apache-2.0 (Nussbaum and Duderstadt 2025)

- Today's demo model is Nomic v2: **open weights, no gate, a GGUF file** for llama.cpp. We use it in English only.

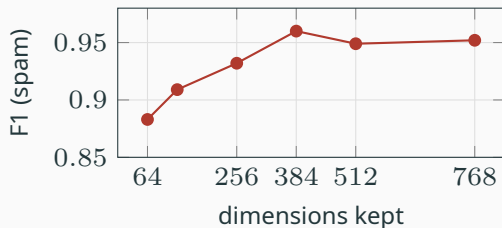
Nomic v2: a mixture of experts



- **Sparse:** 8 expert feed-forward blocks, the router picks 2 per token: 475 M parameters stored, **305 M used** per token (Shazeer et al. 2017; Nussbaum and Duderstadt 2025).

Matryoshka: one vector, several sizes

The first d values of the vector form a valid smaller embedding (Kusupati et al. 2022). Keep them, normalise again. F1 of a linear probe (notebooks/01-spam/07):



- 384 dimensions lose nothing (half the memory: 1.5 KB per message); 256 costs about two points; 64 costs seven.

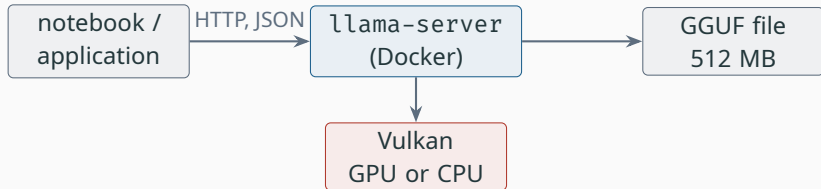
Does the task prefix matter?

Nomic asks for `search_document`: in front of each text. F1 with a 95% bootstrap interval (1,000 resamples of the test set):

prefix	F1	95% interval
<code>search_document</code> :	0.952	[0.924, 0.977]
<code>none</code>	0.960	[0.932, 0.984]
<code>classification</code> :	0.957	[0.926, 0.980]

- The intervals overlap: **for a classification probe the prefix is irrelevant** here. It matters for retrieval (queries versus documents).
- Do not read a difference of 0.005 as an effect without an interval: 1,024 messages, about 130 spam.

Why a server?



- **Separation:** the application sends text and receives vectors; **no PyTorch, no TensorFlow**, no model in the notebook process; several notebooks share one model.
- **Same API as commercial services** (/v1/embeddings, OpenAI-compatible).
- **llama.cpp** (Gerganov and contributors 2023): C++ inference engine for quantised models, on CPUs and (Vulkan) most GPUs.

GGUF and quantisation

- **GGUF**: one file with weights, vocabulary and metadata (pooling type, context length).
- **Quantisation**: weights in 8, 6, 5 or 4 bits (blocks share one scale) instead of 16 or 32.

File (Nomic v2 MoE)	Size
F32	1,908 MB
BF16 / F16	958 MB
Q8_0	512 MB
Q4_K_M	344 MB

- Q4_K_M vs Q8_0: cosine 0.985, F1 0.960 vs 0.952, 48 vs 53 msg/s. Q8_0 vs original: cosine $\geq$ 0.999.

Vulkan: one GPU backend for everybody

- **Vulkan** is a cross-vendor graphics and compute API: AMD, Intel and NVIDIA drivers all implement it. No ROCm, no CUDA, no vendor container.
- The container only needs the device: `/dev/dri`. Without a GPU, the same image runs on the CPU (`--profile cpu`).

Backend (messages/s, batches of 16)	speed
Vulkan, AMD Radeon 860M (integrated GPU)	53
CPU backend, same machine	31
MiniLM 22 M, CPU backend	267

- The MoE model has about 20 times the parameters of MiniLM.

The Docker Compose stack

```
services:
  embed:
    image: ghcr.io/ggml-org/llama.cpp:server-vulkan-b11243 # pinned
    profiles: [gpu]
    devices: ["/dev/dri:/dev/dri"]
    volumes: ["/models:/models:ro"]
    ports: ["127.0.0.1:8080:8080"] # localhost only
    command: -m /models/nomic-embed-text-v2-moe.Q8_0.gguf
              --embeddings -c 512 -np 1 -ngl 99
    healthcheck: {test: ["CMD", "curl", "-sf", "localhost:8080/health"]}
```

- `--embeddings`: embedding endpoint only. `-ngl 99`: all layers on the GPU. A second service `embed-mini` serves MiniLM on port 8082 (CPU). make `llama-up` `llama-check`.

Getting the model

```
python download_model.py --repo nomic-ai/nomic-embed-text-v2-  
moe-GGUF --quant Q8_0  
python download_model.py --repo second-state/All-MiniLM-L6-v2-  
Embedding-GGUF --quant f16
```

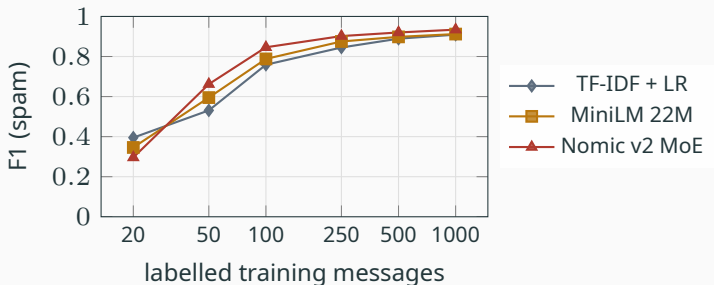
- Nomic (512 MB) and MiniLM (46 MB); make `llama-models` fetches both. From Hugging Face; **resumable** (HTTP Range), **verified** (SHA-256 of the file against the Hub metadata), skips a file that is already good.
- A token for gated repositories comes from the environment (`HF_TOKEN`); it is never stored or printed.
- Pin what you run: image **tag**, model **file**, **checksum**. A latest tag is a supply-chain risk in a security course.

Talking to the server

```
POST http://127.0.0.1:8080/v1/embeddings
{"input": ["search_document: <message>", ...]}
```

- Response: `data[i].embedding`, 768 floats, **already L2-normalised**.
- Batch 16 to 64 messages per request; cache the vectors on disk (the model file name is part of the key).
- About 19 ms per message, batched or not: with `-np 1` the server handles them one after another.

Label efficiency of sentence embeddings



- Notebooks 06 and 07 (10 to 20 repetitions). At 20 labels (about 3 spam) nothing is reliable. From **50** on the embeddings lead (Nomic: +13 points at 50, +3 at 500).
- Fair baselines: an untuned TF-IDF (`min_df=1`) scored 0.05 at 20 labels; tuned, 0.40.

A new campaign: leave one family out

Spam falls in **families**. Train **without** one family, then test on it (recall; false positives below 1% for all models; families of 72 to 172 messages: suggestive, not proven):

held-out family	TF-IDF + LR	MiniLM + LR	Nomic + LR
competition	0.695	0.619	0.762
prize claim	0.988	0.977	0.988
mobile upgrade	0.949	0.906	0.957
account statement	0.949	0.759	0.949
ringtones	0.889	0.931	0.931
chat lines	0.657	0.576	0.808
macro average	0.855	0.795	0.899

The price of meaning

Representation + classifier	F1	ms/msg	Size
TF-IDF + LR	0.944	0.04	about 1 MB
MiniLM (llama.cpp, CPU)	0.952	3.4	22 M parameters
Nomic v2 MoE (llama.cpp, GPU)	0.952	19	475 M, 512 MB file

- With abundant labels the gap is **noise**; an SLM costs **85 to 475 times more** per message.
- **Design rule:** TF-IDF + linear first; an SLM as second stage, when labels are scarce or a new campaign appears.

The ladder in one table

Model	Precision	Recall	F1
Naive Bayes, counts	0.961	0.953	0.957
Logistic regression, TF-IDF	0.983	0.907	0.944
fastText supervised	0.983	0.915	0.948
Word2Vec skip-gram (in-domain) + LR	0.975	0.899	0.935
MiniLM + LR	0.976	0.930	0.952
Nomic v2 MoE + LR	0.976	0.930	0.952

- **All within 0.93 to 0.96.** The clean test score does not separate them. **Cost, label efficiency, new campaigns and robustness** do: the last one is next class.

Labs and practice guide

Notebooks in notebooks/01-spam/:

Notebook	Content
05	Word2Vec and fastText: in-domain vs pre-trained, OOV, few labels
06	SLM embeddings (MiniLM, Nomic) through llama.cpp: probe, few labels, Keras head, cost
07	Nomic v2 MoE through llama.cpp: prefix, Matryoshka, new campaign, throughput
llama/	Docker Compose stack, GGUF download script, smoke test

- **Practice guide** practice/spam/guide_02_02: word vectors, MiniLM, then your own llama.cpp server. Fill-in notebook; solution published afterwards.

- **Vectors** add similarity to a bag of words: **in-domain** with plenty of unlabelled text from the same stream; **pre-trained** only if it covers your vocabulary.
- **Sentence embeddings** lead with few labels or a **new campaign**, at 85 to 475 times the cost.
- Run models as a **service** (llama.cpp, GGUF, Vulkan); **pin** image, file and checksum.
- Report **intervals** and compare with a **fair baseline**.
- **Next class:** we attack every model on the ladder.

Word2Vec objective and sampling

For a corpus of pairs (w, c) with window L and noise distribution $P_n(w) \propto f(w)^{3/4}$ (Mikolov, Sutskever, et al. 2013):

$$J = - \sum_{(w,c)} \left[\log \sigma(\mathbf{v}_w \cdot \mathbf{u}_c) + \sum_{i=1}^k \mathbb{E}_{n_i \sim P_n} \log \sigma(-\mathbf{v}_w \cdot \mathbf{u}_{n_i}) \right]$$

- Frequent words are **sub-sampled**: a word w is dropped with probability $1 - \sqrt{t/f(w)}$, with $t \approx 10^{-5}$: fewer updates on the, to, you.
- Two matrices are learnt, V (word) and U (context); the word vectors used later are the rows of V .

Word2Vec factorises a PMI matrix

At the optimum of the negative-sampling objective (Levy and Goldberg 2014):

$$\mathbf{v}_w \mathbf{u}_c = \text{PMI}(w, c) - \log k, \quad \text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$

- Word2Vec is an implicit **low-rank factorisation** of the shifted word-context association matrix: the neighbourhood structure is a property of the co-occurrence counts, not of the neural network.
- With 65,000 tokens most pairs are seen once or never: the PMI estimates are noisy for rare words, hence the noisy neighbours.

Contrastive loss for sentence embeddings

For a batch of N pairs $(\mathbf{e}_i, \mathbf{e}_i^+)$ with cosine similarity s and temperature τ :

$$\mathcal{L} = -\frac{1}{N} \sum_i \log \frac{\exp(s(\mathbf{e}_i, \mathbf{e}_i^+)/\tau)}{\sum_j \exp(s(\mathbf{e}_i, \mathbf{e}_j^+)/\tau)}$$

- The other $N - 1$ texts of the batch are the **negatives**: larger batches give harder negatives (Oord et al. 2018).
- **Matryoshka loss** (Kusupati et al. 2022): the same loss summed over nested prefixes $\mathbf{e}_{1:d}$, $d \in \{64, 128, \dots, 768\}$, so that each prefix is a good embedding.

Mixture-of-experts routing

For a token state $\mathbf{x}$ and experts $E_1, \dots, E_8$ (Shazeer et al. 2017):

$$\mathbf{y} = \sum_{i \in T} g_i E_i(\mathbf{x}), \quad T = \text{top2}(\mathbf{W}_g \mathbf{x})$$

$$g = \text{softmax}((\mathbf{W}_g \mathbf{x})_T)$$

- Only two feed-forward blocks run: compute per token like a much smaller model, **memory** like the full one (512 MB file).
- A **load-balancing** term keeps the router from sending every token to the same experts.

Quantisation in one formula

A block of weights $\mathbf{w}$ is stored as integers q_i and one scale d (Q8_0: 8 bits):

$$d = \frac{\max_i |w_i|}{127}, \quad q_i = \text{round}(w_i/d), \quad \hat{w}_i = d q_i$$

- The error of every weight is at most $d/2$; with 8 bits it is below 0.4% of the largest weight of the block, and the cosine to the full-precision vector stays at 0.999.
- Fewer bits (Q4_K_M): smaller file, larger error; on this task the cosine to Q8_0 was 0.985 and the F1 unchanged.

Bojanowski, Piotr, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. "Enriching Word Vectors with Subword Information." *Transactions of the Association for Computational Linguistics* 5: 135–46.

Gerganov, Georgi, and contributors. 2023. *Llama.cpp: LLM Inference in C/C++*.

<https://github.com/ggml-org/llama.cpp>.

Bibliography ii

- Joulin, Armand, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. "Bag of Tricks for Efficient Text Classification." *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.
- Kusupati, Aditya et al. 2022. "Matryoshka Representation Learning." *Advances in Neural Information Processing Systems (NeurIPS)*.
- Levy, Omer, and Yoav Goldberg. 2014. "Neural Word Embedding as Implicit Matrix Factorization." *Advances in Neural Information Processing Systems (NeurIPS)*.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean.
2013. *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781.

Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Distributed Representations of Words and Phrases and Their Compositionality." *Advances in Neural Information Processing Systems (NeurIPS)*.

- Nussbaum, Zach, and Brandon Duderstadt. 2025. *Training Sparse Mixture of Experts Text Embedding Models*.
arXiv:2502.07972.
- Oord, Aaron van den, Yazhe Li, and Oriol Vinyals. 2018. *Representation Learning with Contrastive Predictive Coding*.
arXiv:1807.03748.
- Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

- Shazeer, Noam et al. 2017. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer." *International Conference on Learning Representations (ICLR)*.
- Vera, Henrique Schechter, Sahil Dua, Biao Zhang, et al. 2025. *EmbeddingGemma: Powerful and Lightweight Text Representations*. arXiv:2509.20354.